

Lab2 词法分析 实验报告

一、代码实现与词法分析器特性

本实验使用 flexc++ 工具为 Tiger 语言实现了一个完整的词法扫描器。修改的文件为 `src/tiger/lex/tiger.lex` (词法规则文件) 和 `src/tiger/lex/scanner.h` (Scanner 类声明文件)。词法分析器将 Tiger 源代码转换为 token 流, 每个 token 包含类型 (如 `Parser::IF`) 和在源文件中的字符位置。

整个词法分析器基于四个状态的有限状态机, 通过 flexc++ 的排他性起始条件 (`%x COMMENT STR IGNORE`) 声明:

- `INITIAL` : 正常的 token 识别状态
- `COMMENT` : 注释内部状态
- `STR` : 字符串内部状态
- `IGNORE` : 字符串中 `\f__f\` 格式化字符跳过状态

状态转换关系形成了一个清晰的状态图:

```
INITIAL --"/*"--> COMMENT --"*/"--> INITIAL
INITIAL --'"'"--> STR --'"'"--> INITIAL
STR --'\'+空白--> IGNORE --'\ '--> STR
```

这种设计确保进入某一状态后, 只有该状态前缀的规则会被激活, 不同词法上下文完全隔离, 不会产生规则冲突。

• 1.1 注释处理

- 语言规则

Tiger 语言的注释以 `/*` 开始、`*/` 结束, 且支持嵌套。注释可以出现在任意两个 token 之间, 注释中的所有内容不产生任何 token。

- 实现思路

- 由于注释可以嵌套，简单地遇到 `/*` 开始忽略、遇到 `*/` 停止忽略是不可行的。因此，我使用了**整数计数器** `comment_level_` **来追踪当前嵌套了几层注释**：每遇到一个 `/*` 就 +1，每遇到一个 `*/` 就 -1，只有当计数器减到 0 时才真正结束注释。
- 利用 flex++ 的**排他性起始条件** `COMMENT` 将注释状态与正常的 token 识别状态隔离开，确保注释中的内容不会被误识别为其他 token。

- 具体实现规则

规则1：进入注释

```
"/*" {adjust(); comment_level_ = 1; begin(StartCondition_::COMMENT);}
```

在 `INITIAL` 状态下扫描到 `/*` 时触发。

- 首先使用 `adjust()` **推进字符位置计数器**（虽然注释不产生 token，但需要正确记录后续 token 的位置）；
- 接着将 `comment_level_` 设为 1 —— 表示**进入第一层注释**；
- 最后调用 `begin(StartCondition_::COMMENT)` **切换到** `COMMENT` **状态**。切换后，`INITIAL` 状态下的所有规则（关键字、标点符号等）全部失效，只有 `<COMMENT>` 前缀的规则处于激活状态。

规则2：嵌套注释

```
<COMMENT>"/*" {adjust(); comment_level_++;}
```

在 `COMMENT` 状态下再次遇到 `/*`，说明**注释发生了嵌套**。因此只需将层数+1即可。

规则3：关闭注释

```
<COMMENT>"*/" {adjust(); comment_level_--; if(comment_level_ == 0)  
begin(StartCondition_::INITIAL);}
```

遇到 `*/` 时层数-1。只有当**层数减到 0**时，才调用 `begin(StartCondition_::INITIAL)` 返回正常状态。

规则4：注释中的换行

```
<COMMENT>\n {adjust(); errormsg_->Newline();}
```

注释中的换行符不产生 token，但必须调用 `errmsg_->Newline()` 来**更新错误信息模块中的行号**。

- 注意：如果省略此调用，注释之后的所有 token 的行号都会偏小，导致后续的错误报告位置不准确！

规则5：注释中的其他字符

```
<COMMENT>. {adjust();}
```

在正则表达式中匹配除 `\n` 以外的任意单个字符。由于 flex++ 的**最长匹配原则**，`/*` 和 `*/`（2个字符）会优先于 `.`（1个字符）被匹配，所以 `.` 只会匹配到**注释中的普通字符**。

- 每个普通字符被“消耗”后，scanner 继续匹配下一个字符，**整个注释内容被逐字符忽略**。
- 并且没有 `return` 语句，不产生任何 token。

1.2 字符串处理

语言规则

Tiger 语言的字符串字面量由双引号 `"` 括起，内部可以包含普通可打印字符和多种转义序列（`\n` 换行、`\t` 制表符、`\` 反斜线、`\"` 双引号、`\ddd` 三位十进制 ASCII 码、`\^c` 控制字符、`\f__f` 格式化字符跳过）。词法分析器需要**将源码中的转义序列转换为对应的真实字符值**。

实现思路

- 字符串中的转义序列，如 `\n`，是两个字符，但代表的真实值是一个换行符。因此不能直接把源码中双引号之间的文本当作字符串的值，而是需要**逐字符扫描、手动构建真实字符串**。我使用了字符串缓冲区 `string_buf_` 来逐字符收集：遇到普通字符就直接追加，遇到转义序列就转换成对应的真实字符后再追加。
- 利用 flex++ 的**排他性起始条件** `STR` 将字符串内部的处理与正常 token 识别隔离，另外使用 `IGNORE` 状态专门处理 `\f__f` 格式化字符跳过。
- 字符串处理完毕后，通过 `setMatched()` 函数**将构建好的真实字符串设为匹配结果**，供后续使用。
- 此外，整个字符串（包括引号和内部所有字符）只算作一个 `STRING` token，它在输出中的位置应该是**开头引号 `"` 所在的字符位置**。为了实现这一点，我使用了两个不同的**位置更新函数**：

- 进入字符串时（遇到开头的 `"`）调用 `adjust()`，它会同时记录 token 起始位置并推进字符计数器；
- 字符串内部的所有规则（转义序列、普通字符、结束引号等）都调用 `adjustStr()`，它只推进字符计数器，不改变已记录的 token 起始位置。

两个函数的具体区别如下：

```
// scanner.h
inline void Scanner::adjust() {
    errmsg->tok_pos_ = char_pos_; // 设置当前 token 的起始位置
    char_pos_ += length();         // 推进字符位置
}
inline void Scanner::adjustStr() {
    char_pos_ += length();         // 仅推进字符位置，不改变
    tok_pos_
}
```

- 具体实现规则

规则1：进入字符串

```
" {adjust(); string_buf_.clear(); begin(StartCondition_::STR);}
```

遇到 `"` 时：

- 先调用 `adjust()` 记录整个 STRING token 的起始位置；
- 接着清空 `string_buf_` 缓冲区；
- 最后切换到 **STR 状态**。此后 **INITIAL** 状态的所有规则失效，只有 **<STR>** 前缀的规则处于激活状态。

规则2：结束字符串

```
<STR>" {adjustStr(); setMatched(string_buf_);
begin(StartCondition_::INITIAL); return Parser::STRING;}
```

在 STR 状态下遇到匹配的 `"` 时：

- 先调用 `setMatched(string_buf_)` 将手动构建的真实字符串替换掉 flexc++ 自动记录的原始匹配文本，这样 `test_lex.cc` 调用 `scanner.matched()` 时能拿到转义处理后的正确字符串；
- 接着切换回 **INITIAL 状态**；
- 最后返回 `Parser::STRING` 产生一个 STRING token。

规则3：转义 `\n` — 换行符

```
<STR>\\n {adjustStr(); string_buf_ += '\n';}
```

- flex 正则 `\\n` 匹配 Tiger 源码中的两个字符 `\n` ;
- 匹配成功后, 将真实的换行符 `'\n'` (ASCII 10) 追加到 `string_buf_` 中。

注意: `\\` 在 flex 中代表匹配一个字面 `\`, 所以 `\\n` 匹配的实际上是 `\` 和 `n` 两个字符。

规则4: 转义 `\t` — 制表符

```
<STR>\\t {adjustStr(); string_buf_ += '\t';}
```

- flex 正则 `\\t` 匹配源码中的 `\t` 两个字符;
- 匹配成功后, 将真实的制表符 `'\t'` (ASCII 9) 追加到 `string_buf_`。

规则5: 转义 `\\` — 反斜线字符

```
<STR>\\\\ {adjustStr(); string_buf_ += '\\';}
```

- flex 正则 `\\\\` (四个反斜线) 匹配源码中的 `\\` (两个字符) ;
- 匹配成功后, 将一个真实的反斜线字符追加到 `string_buf_`。

规则6: 转义 `\"` — 双引号字符

```
<STR>\\\" {adjustStr(); string_buf_ += '\"';}
```

- flex 正则 `\\\"` 匹配源码中的 `\"` 两个字符;
- 匹配成功后, 将一个双引号字符追加到 `string_buf_`。

注意: 这条规则会优先于结束字符串的 `<STR>\"` 规则匹配, 因为 `\\\"` 匹配2个字符而 `\"` 只匹配1个字符, flex++ 的最长匹配原则保证了正确性。

规则7: 转义 `\\ddd` — ASCII 码

```
<STR>\\[0-9]{3} {adjustStr(); string_buf_ += static_cast<char>  
(atoi(matched().c_str() + 1));}
```

- flex 正则 `\\[0-9]{3}` 匹配 `\` 后跟恰好 3 位十进制数字;
- 匹配成功后, `matched()` 返回如 `\097` (4个字符), `matched().c_str() + 1` 跳过第一个 `\` 得到 `"097"`, `atoi("097")` 转为整数 97, `static_cast<char>(97)` 得到字符 `'a'`, 追加到 `string_buf_` 中。

规则8: 转义 `\\^c` — 控制字符

```
<STR>\\\^. {adjustStr(); string_buf_ += static_cast<char>(matched()[2] - '@');}
```

- flex 正则 `\\\^` 匹配 `\^` 后跟任意一个字符;
- 匹配成功后, `matched()[2]` 取得控制字符标识字母, 减去 `'@'` (ASCII 64) 得到对应的控制字符 ASCII 码。
- 具体控制字符如:
 - `\^@` → `'@'(64) - '@'(64) = 0` (NULL)
 - `\^A` → `'A'(65) - '@'(64) = 1` (SOH)
 - `\^G` → `'G'(71) - '@'(64) = 7` (BEL, 响铃)
 - `\^I` → `'I'(73) - '@'(64) = 9` (HT, 等价于 `\t`)
 - `\^J` → `'J'(74) - '@'(64) = 10` (LF, 等价于 `\n`)
 - `\^[` → `'['(91) - '@'(64) = 27` (ESC, 转义)
 - ...

规则9: 进入格式化字符跳过 (`\f__f\`)

```
<STR>\\[ \t\n\f] {adjustStr(); if(matched()[1] == '\n') errmsg_ ->Newline(); begin(StartCondition_::IGNORE);}
```

Tiger 语言允许使用 `\f__f\` 语法将长字符串分多行书写, 两个 `\` 之间的空白字符会被忽略。

- flex 正则 `\\[ \t\n\f]` 匹配 `\` 后紧跟一个空白字符 (空格、制表符、换行、换页);
- 匹配成功后, `matched()[0]` 是 `\`, `matched()[1]` 是空白字符;
- 如果空白字符是换行符, 需要调用 `errmsg_ ->Newline()` 更新行号;
- 然后切换到 `IGNORE` 状态。

规则10: IGNORE 状态下遇到 `\` — 回到字符串

```
<IGNORE>\\ {adjustStr(); begin(StartCondition_::STR);}
```

在 `IGNORE` 状态下遇到 `\`, 表示格式化跳过区域结束, 切换回 `STR` 状态继续收集字符串字符。

规则11: IGNORE 状态下的空白字符

```
<IGNORE>[ \t\n\f] {adjustStr(); if(matched()[0] == '\n') errmsg_ ->Newline();}
```

在 `IGNORE` 状态下继续跳过空白字符。

- 注意：如果遇到换行符，需要调用 `errmsg->Newline()` 更新行号。虽然 `IGNORE` 中的字符不产生 token，但换行符仍然影响行号计数。

规则12: IGNORE 状态下的非法字符

```
<IGNORE>. {adjust(); errmsg->Error(errmsg->tok_pos_, "illegal token");}
```

在 `\f__f\` 格式化跳过期间，合法的字符只有空白字符和结束的 `\`。如果出现其他字符，通过 `errmsg->Error()` 报告非法 token 错误。

规则13: 普通字符

```
<STR>. {adjustStr(); string_buf_ += matched();}
```

匹配 STR 状态下所有未被前面规则匹配的字符（即普通可打印字符、空格等），直接将 `matched()` 返回的字符追加到 `string_buf_`。

- 注意：由于 flex++ 的最长匹配原则，转义序列（如 `\\n` 匹配2个字符）会优先于此规则（匹配1个字符），所以此规则只处理非转义的普通字符。

1.3 错误处理

词法分析器在三种情况下进行错误处理：

(1) INITIAL 状态下的非法字符

```
. {adjust(); errmsg->Error(errmsg->tok_pos_, "illegal token");}
```

这是 `INITIAL` 状态下所有规则中的最后一条兜底规则：

- 匹配除 `\n` 外的任意单个字符。
- 由于 flex++ 的最长匹配原则和先定义优先原则，所有合法的 token（关键字、标点、标识符、整数等）都会被前面的规则先匹配到。
- 因此只有 Tiger 语言中没有定义的字符（如 `@`、`#`、`$`、`!` 等）才会落到这条兜底规则。
- 报错后 scanner 不会停止，而是消耗掉这个非法字符继续扫描，可以一次性发现多个词法错误。

(2) IGNORE 状态下的非法字符

即上述1.2节中字符串处理的规则12，在 `\f__f\` 格式化跳过期间遇到**非空白、非反斜线字符**时报错。

(3) 未关闭的注释或字符串

在**文件结尾 (EOF)** 时检查，详见下节1.4。

• 1.4 文件结尾处理

当 scanner 读到文件末尾 (EOF) 时，如果当前仍处于 `COMMENT` 或 `STR` 状态，说明源码中存在**未关闭的注释或字符串**，需要报错。

为此，我在 `scanner.h` 的 `postCode()` 回调函数中实现了 EOF 检查：

```
// scanner.h
inline void Scanner::postCode(PostEnum_ type) {
    if (type == PostEnum_::END) {
        if (startCondition() == StartCondition_::COMMENT) {
            errmsg_>Error(errmsg_>tok_pos_, "unexpected end of file in
comment");
        }
        if (startCondition() == StartCondition_::STR) {
            errmsg_>Error(errmsg_>tok_pos_, "unexpected end of file in
string");
        }
    }
}
```

- `postCode()` 是 flex++ 框架在**每次 `lex()` 返回前自动调用**的函数，它的参数 `type` 指示了返回的原因——当 `type == PostEnum_::END` 时，表示 scanner 已经到达了**输入文件的末尾 (EOF)**。
 - 此时，通过调用 `startCondition()` 获取**当前所处的起始条件状态**：
 - 如果仍在 `COMMENT` 状态，说明有 `/*` 缺少对应的 `*/`；
 - 如果仍在 `STR` 状态，说明有开头的 `"` 缺少对应的结束 `"`。
 - 正常情况下，EOF 时应处于 `INITIAL` 状态。
 - 最后根据当前状态调用 `errmsg_>Error()` 报告相应的错误信息，并且使用 `tok_pos_` 来指示**错误位置**。
-

• 1.5 其他有趣特性

- (1) 错误恢复的"继续扫描"策略

词法分析器的兜底规则 `. {adjust(); errmsg->Error(...);}` 在报错后**不返回任何 token、不调用 `exit()`**，scanner 会消耗掉非法字符后继续扫描下一个字符。

这体现了编译器设计的重要哲学：**一次运行发现尽可能多的错误**，而不是遇到第一个错误就停止。用户修复一个错误后重新编译，可能又发现下一个错误——因此编译器需要能够**一次性报告所有词法错误**，这样开发效率会大幅提升。

- 这种策略在词法层面是可行的，因为**词法错误是局部的**——一个非法字符（如 `@`）不影响它前后字符的识别。scanner 跳过该字符后，后续的 token 仍然可以被正确识别。
- 对比 parser 的错误处理（`parser.h` 中 `error()` 直接调用 `exit(1)`）：**语法错误**会导致 parser 的状态机进入不确定状态，**后续所有 token 的解析都可能出错**，因此选择直接终止。

这反映了词法层和语法层在错误恢复能力上的本质差异：**词法分析处理的是字符级别的独立单元**，错误不会"传染"；而语法分析处理的是**token 之间的结构关系**，一个错误会破坏整个解析上下文。

- (2) 匹配优先级机制

flex++ 的规则匹配遵循两条核心原则：**最长匹配原则**和**先定义优先原则**，它们在不同场景下配合工作，共同保证词法分析的确定性：

- **最长匹配原则**：当多条规则能匹配同一位置的输入时，**匹配最多字符的规则优先**。
 - 输入 `:=` 时，规则 `":="` 匹配2个字符，规则 `":"` 匹配1个字符，因此无论定义顺序如何，`:=` 都会被正确识别为 ASSIGN。同理，`<=`、`>=`、`<>` 也因为更长而自动优先于 `<`、`>`、`=`。
 - 在字符串处理中，`<STR>\\\"` 匹配2个字符，自动优先于 `<STR>\"` 匹配1个字符，定义顺序无关。
 - 输入 `whileabc` 时，标识符规则匹配8个字符，而 `"while"` 只能匹配5个字符，因此 `whileabc` 被正确识别为 ID 而非 WHILE。
- **先定义优先原则**：当多条规则匹配**相同长度**的输入时，**先定义的规则优先**。
 - 例如输入 `while` 时，精确字符串 `"while"` 和正则表达式 `{letter}({letter}|{digit}|_)*` 都能匹配5个字符。由于 `"while"` 规则定义在标识符规则之前，**关键字规则胜出**，返回 WHILE 而非 ID。
 - 因此我在 `tiger.lex` 中**将保留字的规则都定义在标识符规则之前**——如果顺序颠倒，所有关键字都会被错误地识别为标识符。

- 两条原则的配合：**先比较匹配长度，长度相同时再比较定义顺序**。这使得 flexc++ **无需维护关键字哈希表**（传统 lex 的做法），关键字识别在 DFA 中自然完成，**没有额外的运行时开销**。

- (3) 词法分析器的性能特征

flexc++ 将所有语法规则编译为一个**确定性有限自动机 (DFA)**，这保证了词法分析的时间复杂度严格为 $O(n)$ ，其中 n 是源文件的字符数。

- DFA 的核心特性：对于任意输入字符，当前状态只有**唯一确定的转移边**，**无需尝试多条路径或回溯**。即使 Tiger 的语法规则包含嵌套注释、多种转义序列、格式化字符跳过等复杂特性，DFA 仍然能够**单遍扫描完成识别**，每个字符只被读取一次。
- 对比其他方案：
 - **递归下降词法分析器**：在识别多字符运算符时可能需要**回溯**（例如读到 `<` 后尝试匹配 `<=`，失败则回退），最坏情况下时间复杂度可达 $O(n^2)$ 。
 - **NFA 模拟器**：需要维护**多个活跃状态集合**，时间复杂度为 $O(n \cdot s)$ (s 是状态数)。

可见，flexc++ 的 DFA 方案是典型的**"用空间换时间"**——生成的状态转移表可能包含数千个状态，但**运行时性能极佳**，因此词法分析完全不是编译器的性能瓶颈。

二、实现过程中的挑战与 Bug 修复

• 挑战

- (1) 理解 flexc++ 起始条件机制

排他性起始条件 (`%x`) 的核心是**状态隔离**——进入某个状态后，只有带对应前缀的规则会被激活，`INITIAL` 状态下的所有规则全部失效。这意味着注释内部的 `*`、`/` 不会触发运算符规则，字符串内部的关键字也不会被识别为 token。

与之对应的**包含性起始条件** (`%s`) 则不会屏蔽无前缀的规则，不适合用于注释和字符串的处理。

我在实现时的主要困惑在于**先需要理解排他性与包含性起始条件的区别**，然后再分析清楚 `begin()` 切换状态后哪些规则仍然有效、哪些被屏蔽，这样问题才能迎刃而解。

- (2) 嵌套注释的正确处理

最初我的直觉是用一个布尔标志（在注释中/不在注释中）来处理注释，但用 `/* outer /* inner */ still comment */` 这样的测试用例验证时发现遇到第一个 `*/` 就错误退出了注释。

意识到 Tiger 支持嵌套注释后，我改用整数计数器 `comment_level_` 追踪嵌套深度，只有层数归零才真正结束注释。

- (3) 字符串转义的双重转义问题

flex 正则表达式和 Tiger 语言各自有一层转义，两层叠加容易出错。

例如，Tiger 源码中的 `\`（反斜线转义）在 flex 正则中需要写成 `\\` —— 每个字面 `\` 都需要一个额外的 `\` 来转义。类似地，`"`（双引号转义）在 flex 中写成 `\"`，`^`（控制字符）写成 `^`。

我在实现时容易少写或多写反斜线导致匹配失败，处理的方式是分两步思考：先确定 Tiger 源码中实际出现的字符序列，再对其中每个 flex 特殊字符进行正则层面的转义。

- (4) 字符串 token 位置的正确记录

整个字符串（从开头 `"` 到结尾 `"`）只产生一个 STRING token，其位置应该是开头引号的字符位置。

框架提供了两个位置更新函数：`adjust()` 会同时设置 `tok_pos_`（token 起始位置）并推进 `char_pos_`（字符计数器），而 `adjustStr()` 只推进 `char_pos_` 不改变 `tok_pos_`。

实现时我遇到的陷阱是：如果在 STR 状态内部的规则中误用了 `adjust()`，`tok_pos_` 会被更新为字符串中间某个位置，覆盖掉进入字符串时在开头引号处正确设置的值，导致 STRING token 的位置偏移。

因此正确做法是：==只有进入字符串的 `"` 规则使用 `adjust()`，STR 状态下所有其他规则都使用 `adjustStr()`。

- (5) `\f__f\` 格式化字符跳过的实现

Tiger 允许在字符串中用 `\f__f\`（两个 `\` 之间包裹空白字符）来跨行书写长字符串，中间的空白被忽略。这个特性的难点在于它发生在字符串内部，但行为又不同于普通字符串字符 —— 需要忽略而非收集。

我最初考虑在 `STR` 状态内用标志变量处理，但发现这样会让 `STR` 状态的规则变得复杂且容易出错。最终我选择使用**第三个独立状态 `IGNORE`**：在 `STR` 中遇到 `\` 后跟空白时切换到 `IGNORE`，在 `IGNORE` 中只接受空白字符（跳过）和结束的 `\`（回到 `STR`），其他字符报错。

这种**三状态设计**让每个状态的职责单一明确。此外还需要注意 `IGNORE` 中的**换行符**虽然不产生字符，但仍需调用 `Newline()` **更新行号**，否则后续 token 的行号会偏移。

• Bug 修复记录

- Bug 1: 框架代码与 flexc++/bisonc++ 版本不兼容

- **现象**：编译时出现大量错误，如 `'PostEnum__' has not been declared`、`no declaration matches 'int Scanner::executeAction_(size_t)'`、`'d_val__' was not declared in this scope` 等。
- **排查**：注意到报错集中在**双下划线命名**的标识符上，于是查看 flexc++ 实际生成的 `scannerbase.h` 和 bisonc++ 生成的 `parserbase.h`，发现生成代码中的内部名称均为**单下划线**（如 `PostEnum_`、`executeAction_`），而框架提供的 `scanner.h` 和 `parser.h` 中使用的是**双下划线**（如 `PostEnum__`、`executeAction__`），判断是版本不匹配。
- **原因**：框架代码对应的是旧版本的 flexc++/bisonc++，而实验环境安装的版本生成的代码使用单下划线命名。
- **修复**：将 `scanner.h` 和 `parser.h` 中**所有双下划线改为单下划线**。

- Bug 2: 字符串内部误用 `adjust()` 导致 token 位置偏移

- **现象**：运行测试用例时，包含转义序列的字符串输出的 `STRING` token 位置不是开头 `"` 的位置，而是**偏移到了字符串内部**，与参考输出不匹配。
- **排查**：对比参考输出发现只有包含转义序列的字符串位置不对，简单字符串是正确的。检查 `STR` 状态下各规则的动作代码，发现**部分转义处理规则中调用了 `adjust()` 而非 `adjustStr()`**。阅读 `scanner.h` 中两个函数的实现后，确认 `adjust()` 会将 `tok_pos_` 更新为当前字符位置，从而覆盖了进入字符串时在开头引号处设置的值。
- **修复**：进入字符串的 `"` 规则使用 `adjust()`（设置 token 起始位置），`STR` 状态下的**所有其他规则**都使用 `adjustStr()`（仅推进 `char_pos_`）。

- Bug 3: IGNORE 状态中换行符未更新行号

- **现象**：当字符串使用 `\f____f\` 格式化跳过跨行时，后续 token 的行号比参考输出偏小。
- **排查**：对比输出发现行号偏差恰好等于 `\f____f\` 跳过区域中换行符的数量，定位到 IGNORE 状态的空白字符处理规则。
- **原因**：最初 IGNORE 状态的规则只有 `<IGNORE>[ \t\n\f] {adjustStr();}`，没有对换行符调用 `errmsg->Newline()`。
- **修复**：在 IGNORE 状态的空白字符规则中增加换行检测：`if(matched()[0] == '\n') errmsg->Newline()`。同样，进入 IGNORE 状态的规则也需要检查 `matched()[1]`。